SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **V.Jayaprakash (192311057))** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

V.Jayaprakash

Annexure A

Constraint-Based Problem Solving

1. **A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:**

   Constraints:

   i. **D1 cannot work Night shift**

   ii. **D2 must work before D3 (Morning < Afternoon < Night)**

   iii. **Only one doctor per shift**

   iv. **D3 cannot work Morning**

   v. **All doctors must be assigned exactly one shift**

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:

Problem formulation

Variables: D1, D2, D3 (each doctor). Domain of each variable: {Morning (M), Afternoon (A), Night (N)} with the order M < A < N. Constraints: C1 - D1 cannot work Night; C2 - D2 must work before D3 (shift of D2 < shift of D3); C3 - only one doctor per shift (all values different); C4 - D3 cannot work Morning; C5 - every doctor gets exactly one shift. Variable order used: D1, D2, D3. Value order used: M, A, N.

Backtracking search - intermediate steps and constraint checks

Step	Partial assignment tried	Constraint check	Decision
1	D1 = Morning	C1 OK (D1 is not Night)	Accept; go to D2
2	D1 = M, D2 = Morning	C3 violated (Morning already used by D1)	Reject; try next value
3	D1 = M, D2 = Afternoon	C3 OK; C2 pending (D3 must be after Afternoon)	Accept; go to D3
4	D1 = M, D2 = A, D3 = Morning	C3 violated; C4 violated (D3 cannot be Morning); C2 violated	Reject
5	D1 = M, D2 = A, D3 = Afternoon	C3 violated (Afternoon used by D2); C2 violated (A is not before A)	Reject
6	D1 = M, D2 = A, D3 = Night	C1 OK, C2 OK (A < N), C3 OK, C4 OK, C5 OK	**SOLUTION 1 FOUND**

Step	Partial assignment tried	Constraint check	Decision
7	Backtrack to D2: D1 = M, D2 = Night	C3 OK, but C2 needs D3 after Night, which is impossible. D3 = M: C3, C4 violated. D3 = A: C2 violated. D3 = N: C3 violated	Dead end; backtrack to D1
8	D1 = Afternoon	C1 OK	Accept; go to D2
9	D1 = A, D2 = Morning	C3 OK	Accept; go to D3
10	D1 = A, D2 = M, D3 = Morning	C3 violated; C4 violated	Reject
11	D1 = A, D2 = M, D3 = Afternoon	C3 violated (Afternoon used by D1)	Reject
12	D1 = A, D2 = M, D3 = Night	C1 OK, C2 OK (M < N), C3 OK, C4 OK, C5 OK	**SOLUTION 2 FOUND**
13	D1 = A, D2 = Afternoon	C3 violated	Reject
14	D1 = A, D2 = Night	C2 cannot be satisfied (no shift after Night); every D3 value fails	Dead end; backtrack to D1
15	D1 = Night	C1 violated (D1 cannot work Night)	Reject; whole branch pruned

Final valid schedule (first solution found by backtracking)

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Verification: C1 - D1 works Morning, not Night (OK). C2 - D2 (Afternoon) works before D3 (Night) (OK). C3 - one doctor per shift (OK). C4 - D3 works Night, not Morning (OK). C5 - each doctor has exactly one shift (OK).

Note: continuing the search shows that exactly one other valid schedule exists (D1 - Afternoon, D2 - Morning, D3 - Night); the first complete solution found by the chosen variable and value ordering is the one reported above.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. **Movement allowed: Up, Down, Left, Right**
ii. **Cost of each move = 1**
iii. **Cannot pass through obstacles**
iv. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Answer:

Environment used: The question paper does not print the grid figure, so the following 5 x 5 grid is assumed. Cells are written as (row, column); S = (1,1) is the Start, G = (5,5) is the Goal, and X marks obstacles at (1,3), (2,3), (3,2), (4,4) and (5,3). The same procedure applies to any other layout.

	1	**2**	**3**	**4**	**5**
1	S	.	X	.	.
2	.	.	X	.	.
3	.	X	.	.	.
4	.	.	.	X	.
5	.	.	X	.	G

Method

- Search tree is expanded level by level as in BFS. Every move costs 1, so the cost g(n) equals the number of moves from S. The frontier is kept as a priority queue ordered by f(n) = g(n) + h(n), where h(n) is the Manhattan distance |row - 5| + |col - 5| to G. Ties are broken by smaller h, then by the order of insertion.
- Children are generated in the order Up, Down, Left, Right. Obstacles, cells outside the grid and already-expanded cells are skipped. Initial h(S) = 4 + 4 = 8.
- The goal test is applied when a node is removed from the queue; at that moment its g value is the optimal cost, because the Manhattan heuristic never overestimates (every move costs at least 1).

Step-by-step node expansion and priority queue updates

Step	**Node expanded [g+h=f]**	**Children generated / updated [g+h=f]**	**Priority queue after the step (best first)**
1	(1,1) [0+8=8]	(2,1) [1+7=8], (1,2) [1+7=8]	(2,1) [1+7=8], (1,2) [1+7=8]
2	(2,1) [1+7=8]	(3,1) [2+6=8], (2,2) [2+6=8]	(3,1) [2+6=8], (2,2) [2+6=8], (1,2) [1+7=8]
3	(3,1) [2+6=8]	(4,1) [3+5=8]	(4,1) [3+5=8], (2,2) [2+6=8], (1,2) [1+7=8]
4	(4,1) [3+5=8]	(5,1) [4+4=8], (4,2) [4+4=8]	(5,1) [4+4=8], (4,2) [4+4=8], (2,2) [2+6=8], (1,2) [1+7=8]
5	(5,1) [4+4=8]	(5,2) [5+3=8]	(5,2) [5+3=8], (4,2) [4+4=8], (2,2) [2+6=8], (1,2) [1+7=8]
6	(5,2) [5+3=8]	None (all neighbours are obstacles or already expanded / queued)	(4,2) [4+4=8], (2,2) [2+6=8], (1,2) [1+7=8]
7	(4,2) [4+4=8]	(4,3) [5+3=8]	(4,3) [5+3=8], (2,2) [2+6=8], (1,2) [1+7=8]
8	(4,3) [5+3=8]	(3,3) [6+4=10]	(2,2) [2+6=8], (1,2) [1+7=8], (3,3) [6+4=10]

Step	Node expanded [g+h=f]	Children generated / updated [g+h=f]	Priority queue after the step (best first)
9	(2,2) [2+6=8]	None (all neighbours are obstacles or already expanded / queued)	(1,2) [1+7=8], (3,3) [6+4=10]
10	(1,2) [1+7=8]	None (all neighbours are obstacles or already expanded / queued)	(3,3) [6+4=10]
11	(3,3) [6+4=10]	(3,4) [7+3=10]	(3,4) [7+3=10]
12	(3,4) [7+3=10]	(2,4) [8+4=12], (3,5) [8+2=10]	(3,5) [8+2=10], (2,4) [8+4=12]
13	(3,5) [8+2=10]	(2,5) [9+3=12], (4,5) [9+1=10]	(4,5) [9+1=10], (2,5) [9+3=12], (2,4) [8+4=12]
14	(4,5) [9+1=10]	(5,5) [10+0=10]	(5,5) [10+0=10], (2,5) [9+3=12], (2,4) [8+4=12]
15	(5,5) [10+0=10]	Goal test passed - stop	(empty) - goal reached

Remarks: the cells (1,2), (2,2), (5,1) and (5,2) are expanded only because their f value (8) is lower than the cost of the real way round; they are dead ends enclosed by obstacles. After step 10 the queue contains only (3,3) with f = 10, so the search is forced to use the corridor (4,3) - (3,3). In total 15 nodes were expanded.

Optimal path and cost

Path: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (4,2) -> (4,3) -> (3,3) -> (3,4) -> (3,5) -> (4,5) -> (5,5).

Moves: Down, Down, Down, Right, Right, Up, Right, Right, Down, Down (10 moves).

Optimal cost = 10 (10 moves x cost 1). The Manhattan lower bound is 8, but the obstacles at (3,2), (5,3) and (4,4) force a detour of 2 extra moves. Because the heuristic is admissible and consistent, no cheaper path exists.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- **The robot can move up, down, left, right**
- **Some cells are blocked (obstacles) and cannot be traversed**
- **The robot has limited sensing capability (can only observe adjacent cells)**
- **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- **The robot must minimize total cost while ensuring safe navigation**
- **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

Answer:

(a): Effect of the constraints on decision-making

Constraint	How it affects the robot's decision-making
Moves: up, down, left, right	The state space is the set of grid cells and every state has at most 4 actions (branching factor 4). No diagonal shortcuts are possible, so path length is measured in Manhattan steps.
Blocked cells (obstacles)	Blocked cells remove states/actions from the problem. A route that looks short may be impossible and force a detour, so the robot must keep a record of obstacles it has discovered.
Limited sensing (adjacent cells only)	The environment is partially observable. The robot cannot compute a complete plan in advance; it must keep an internal map (belief state), treat unseen cells as unknown, and decide using only what it currently knows.
Move cost 1, risky zone +2	Step costs are not uniform: a normal move costs 1, entering a risky cell costs 1 + 2 = 3. The shortest path (fewest moves) is therefore not always the cheapest, so a cost-sensitive search is needed. A risky cell is worth entering only when every safe alternative costs more than 2 extra units; if the costs are equal the safe route is chosen.
Minimise cost while ensuring safe navigation	Two goals compete: low cost and low risk. Safety is built into the cost function (risk penalty) and into the hard rule never to enter blocked cells. The robot trades extra distance against risk using total path cost.
Dynamic knowledge update (online search)	Planning and acting are interleaved. After every move the robot senses, updates its map, and re-plans if its current plan is no longer valid. Plans are only as good as the current knowledge, so they are short-term and revised continually.

Priority of constraints: (1) hard constraints - allowed moves and blocked cells (a violation makes a plan invalid); (2) safety - risky-zone penalty; (3) minimum total cost; (4) knowledge update, which keeps the first three accurate as new information arrives.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

(b): Solution approach - Online Uniform Cost Search (UCS) agent with re-planning

Uniform Cost Search expands the node with the lowest path cost g(n) first, so it finds the cheapest route when step costs differ (1 or 3). Because the map is not fully known in advance, UCS is run repeatedly inside an online agent loop: plan on the known map, move one step, sense, update, re-plan.

Steps involved:

1. **Formulate the problem.** State = robot cell (plus the known map). Initial state = S. Goal test = robot is at G. Actions = Up, Down, Left, Right to a non-blocked neighbour. Step cost = 1, or 3 if the cell entered is risky.
2. **Initialise the knowledge base.** Store S and G. All other cells are unknown. Sense the cells adjacent to S and record which are free, blocked or risky.

3. **Plan with UCS on the known map.** Unknown cells are assumed free and safe (optimistic assumption, cost 1). Keep a priority queue ordered by g(n); remove the cheapest node, test for goal, and insert its non-blocked neighbours with g(n) + step cost.
4. **Act.** Execute only the first move of the cheapest plan found.
5. **Sense and update.** Observe the four adjacent cells and update the map (new obstacles, new risky zones).
6. **Re-plan if needed.** If the next cell of the plan is now blocked, or a risky cell has made the plan more expensive, discard the old queue and run UCS again from the robot's current cell.
7. **Repeat** steps 3 to 6 until the robot reaches G (survivor located). If the queue becomes empty, no safe known route exists and the robot reports failure or explores unknown cells.

Pseudo-code

```
robot = S; KB = {S, G}; sense(robot, KB)
while robot != G:
   plan = UCS(robot, G, KB)       # cost: 1 per move, +2 for risky cell
   if plan is empty: return FAILURE
   next = plan[0]
   move(robot, next)              # act: one step only
   sense(robot, KB)               # observe adjacent cells, update KB
return SUCCESS                    # survivor reached
```

Illustration of the cost-based decision rule

Route	Moves	Cost calculation	Total cost	Decision
A: through one risky cell	4	3 normal moves x 1 + 1 risky move x 3	6	Candidate
B: safe detour	6	6 normal moves x 1	6	Chosen (equal cost, safer)
C: long safe detour	8	8 normal moves x 1	8	Rejected - A is cheaper

c) Demonstrate how the robot applies AI concepts such as:

- **Intelligent agents**
- **Rationality**
- **Environment type**

Answer (c): Application of AI concepts

1. Intelligent agent. The rescue robot is a model-based, goal-based agent with utility (cost) considerations: it keeps an internal map, has the goal of reaching the survivor, and compares actions by cost. Its PEAS description is:

PEAS element	Rescue robot
Performance measure	Reach G with minimum total cost, avoid risky zones where possible, avoid collisions with obstacles
Environment	Disaster-affected building represented as a grid of rooms with blocked cells and risky zones
Actuators	Wheels / drive motors for Up, Down, Left, Right moves

PEAS element	Rescue robot
Sensors	Adjacent-cell sensors (obstacle / risk detection), position sensor (odometry)

Agent function: percept sequence -> action. Agent program: the loop sense -> update map -> plan with UCS -> move one step.

2. Rationality. A rational agent selects the action expected to maximise its performance measure, given the percepts received so far and its built-in knowledge. The robot is rational because at every step it chooses the action on the cheapest known route to G, it avoids risky cells unless every safe route costs more, and it gathers information (sensing) before committing to the next move. Rationality is not omniscience: the robot cannot know unseen cells, so it acts optimally with respect to what it knows and corrects itself when new percepts arrive.

3. Environment type.

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed
Number of agents	Single agent	Only the robot acts; obstacles do not act
Determinism	Deterministic	A move action leads to the intended cell; uncertainty comes only from missing knowledge
Episodic / sequential	Sequential	Each move affects the future cost and options
Static / dynamic	Static (semi-dynamic if more collapse occurs)	The map is assumed to stay the same while the robot decides; new information is handled by re-planning
Discrete / continuous	Discrete	Grid cells and four fixed actions
Known / unknown	Unknown at start	The map is learned during exploration, which is why online search is used

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer (d): Justification of the chosen approach

- **Handles different step costs.** The risky-zone penalty makes costs 1 or 3, so BFS (which minimises the number of moves, not cost) can return a more expensive and less safe route. UCS is optimal for any positive step costs.
- **Works with partial observability.** The map is unknown at the start, so an offline plan cannot be computed. An online agent that senses, updates its knowledge and re-plans can operate with only local information.
- **Complete and optimal on the known map.** The grid is finite and every step costs at least 1, so UCS always finds a path if one exists in the known map and returns the cheapest one. Re-planning keeps the plan valid when new obstacles appear.
- **Supports safe navigation.** Risk is converted into cost, so avoiding risky zones is part of the minimisation, while blocked cells are never entered.
- **Compared with alternatives.** DFS and greedy search are fast but neither optimal nor cost-aware, and they may lead the robot into risky regions. A* with the Manhattan heuristic is a natural efficiency improvement, because the goal position is known and

the heuristic never overestimates; it can replace UCS inside the same online loop. UCS is chosen as the basic strategy because it needs no heuristic and is easy to justify for cost-based safety.

Conclusion: An online Uniform Cost Search agent with re-planning is the most suitable approach because it is cost-aware, safe, and able to operate with limited sensing in an unknown environment

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided